面向异构 GPU 流水线推理的路径空间控制

摘要

异构 GPU 资源池可以通过层段复制为流水线推理提供多条执行路径。现有系统通常将这些路径视为服务容量，路径越多，可调度的能力越大。本文发现这一假设在交互式服务中并不成立。Decode 批处理要求请求汇聚到同一组副本，慢链路与瓶颈设备又要求请求分流；额外路径是否有益，取决于分流发生的位置、活跃请求对路径的持续占用以及长传输对慢链路的占用。基于这一观察，本文提出 RCAPS，将可执行路径空间作为受控资源，在部署、选路与传输三个时间点分别加以控制。部署时采用前段共享、后段按需复制的放置，并只在慢链路附近开放绕行；选路时将活跃请求的持续负载折算为路径代价，在层进度执行图上以动态规划求出最小代价路径；传输时将慢链路上的 Prefill 激活分块发送，在块边界优先发送 Decode 消息。基于 LLaMA-70B 与 24 张异构 GPU 的离散事件模拟表明，在统一执行规则并独立调优基线的条件下，相较于各负载点上该指标最优的 Helix 基线，RCAPS 将 P95 TPOT 降低 51.6%～81.6%，将 P95 TTFT 降低 40.1%～66.1%。

关键词：异构 GPU；大模型推理；流水线推理；路径空间控制；持续负载感知路由

1 引言

大语言模型（LLM）[1-3]已成为对话与代码生成等交互式应用的基础设施，这类服务对首 token 延迟（TTFT）与相邻输出 token 间隔（TPOT）的尾部表现高度敏感[4]。受高端 GPU 供给与成本的限制，推理服务越来越多地部署在混合多代 GPU、跨越多个地域的资源池上[5-7]。在 LLaMA-70B 上，一张 A100 可驻留 12 层，一张 T4 只能驻留 4 层；集群内链路可达 GB/s 量级，跨地域链路平均只有约 12.5 MB/s。

流水线推理按层切分模型，同一层段存在多个副本时，请求可以在副本之间切换，形成多条可执行路径[8,9]。HexGen[10]、ThunderServe[11]与 Hetis[12]通过放置与并行策略决定各设备承担的计算份额；Helix[5]为每个请求分配独立的流水线，用最大流按稳态流量比例分配请求，Parallax[13]为每个请求选择逐层延迟最小的跨副本执行链。后两项工作将可执行路径视为服务容量。Helix 指出，可选流水线的数目等于执行图中从源到汇的路径数，路径越多，调度越能贴合各节点与连接的容量[5]。从吞吐的角度看，这一判断是合理的。

然而在交互式服务中，尾延迟同时受两种方向相反的需求支配。一是汇聚。Decode 单步耗时在小批量范围内几乎不随批量变化（表 1），12 个活跃请求集中在一个 L4 副本上时每层每步约需 7 ms 的 GPU 时间，分到三个副本则需约 21 ms。二是分流。750 个 token 的 Prefill 激活在 12.5 MB/s 的跨地域链路上需要约 1 s 才能发完，请求只有绕开这类链路才能避免排队。两种需求的强弱随网络而变，开放全部合法路径后，单集群的 P95 TPOT 上升 10.5%，三集群反而下降 15.2%（第 2.2 节）。可见路径数量本身无法预测尾延迟。

现有系统只决定开放多少路径、按什么比例分配请求，没有控制汇聚与分流之间的取舍，这体现在三个环节：对全部合法的跨副本连接同等对待，使远离慢链路的分流打散了 Decode 批处理；只依据稳态流量或瞬时延迟选路，无法反映请求在此后数百个 Decode 步中对路径的持续占用；整条发送长 Prefill 激活，使其在秒级时间内独占慢链路。本文的关键洞察是，分流的价值随位置与时间而变，可执行路径空间应当作为受控资源来管理。据此，本文提出 RCAPS，在部署、选路与传输三个时间点分别控制分流位置、持续负载与链路占用。RCAPS 沿用 Helix[5]的层进度连接规

则构造执行图，层图上的动态规划选路也已见于 Parallax[13]；本文的贡献在于在该执行图上构造受控的路径空间，并为请求的整个生命周期计价。

实现这一思路面临三个挑战。第一，尾延迟无法在部署时直接计算，而 GPU 到副本的分配方式随 GPU 数量组合增长；RCAPS 以前段共享、后段按需复制的结构约束缩小搜索空间，并只在慢链路两端开放绕行。第二，持续负载与当前等待的性质不同，而完整路径的数目随副本数成倍增长，单集群部署已有 4788 条；RCAPS 将持续负载折算为以秒计的可加代价，使选路成为执行图上的最短路问题，开销与可用边数成线性。第三，Prefill 激活必须完整到达下游才能参与计算；RCAPS 只切分传输，在块边界先发送已就绪的 Decode 消息。

我们在 Helix 开源的离散事件模拟器上实现 RCAPS，并在由 24 张异构 GPU 组成的单集群与三集群拓扑上进行评估。在统一执行规则并独立调优基线的条件下，相较于各负载点上该指标最优的 Helix 基线，RCAPS 将 P95 TPOT 降低 51.6%～81.6%，将 P95 TTFT 降低 40.1%～66.1%，其中分段共享放置贡献了主要收益，代价是更多的 GPU 计算。

本文的主要贡献如下：

（1）问题分析。揭示异构多路径流水线中汇聚与分流的矛盾，并将其归结为三个未受控制的环节（第 2 节）。

（2）路径空间构造。提出分段共享与复制放置，以及只在慢链路附近开放绕行的路径空间裁剪（第 3.2～3.3 节）。

（3）选路与传输控制。提出持续负载感知的层序动态规划选路，以及只切分传输、不切分计算的 Prefill 传输分块（第 3.4～3.5 节）。

（4）实现与评估。在统一执行规则、独立调优基线的条件下完成端到端比较、放置与路由交叉实验、机制消融及 TCP 限速链路实验，并量化尾延迟与计算量之间的取舍（第 4 节）。

2 背景与动机

2.1 流水线推理与异构资源

大语言模型由若干 Transformer 层[1]顺序组成，推理分为两个阶段。Prefill 阶段一次性处理整个输入并建立 KV 缓存，属于计算密集型；Decode 阶段逐个生成输出 token，单步计算量小、受显存带宽限制，但在请求生命周期内反复执行[14,15]。已有工作通过分块预填充[16]、阶段分离[17-19]与以 KV 缓存为中心的架构[20]协调两个阶段的资源需求。70B 规模的模型无法放入单卡，需要按层切分形成流水线[21-23]，自动并行与推理框架进一步支持了更灵活的切分方式[24,25]。当同一层段存在多个副本时，请求可以在副本之间切换，形成多条可执行路径。

本文采用固定路径的流水线执行模型。请求的 KV 缓存分布在所选路径的各个 GPU 上，服务过程中不迁移，因此路径一经确定，后续每个 Decode 步都必须回到同一组 GPU 执行，直到生成全部输出并释放 KV 缓存。本文将尚未完成 Decode 的请求称为活跃请求。与只占用一次资源的任务不同，活跃请求在整个 Decode 过程中反复使用路径上的计算与通信资源，对这些资源构成持续负载。

异构资源池中，设备的显存与计算速度相差悬殊[12,26]。表 1 给出 LLaMA-70B 在三类 GPU 上的逐层剖析数据。A100 最多驻留 12 层，L4 与 T4 分别只能驻留 7 层与 4 层；处理 1000 个输入 token 时，T4 的 Prefill 每层耗时约为 A100 的 7.4 倍。Decode 单步耗时在批量 2～12 之间变化不超过 9%，因此同一层上同时执行的 Decode 越多，每个 token 分摊的 GPU 工作越少，请求汇聚在同一副本上有直接的计算收益。网络差异更大：跨集群链路平均带宽约 12.5 MB/s、单向时延约 50

ms，750 个 token 的 Prefill 激活约 12.3 MB，需要约 1 s 才能发完，而一个 Decode 步的激活只有 16 KB，二者相差近三个数量级。前一组数字说明请求为什么需要汇聚，后一组数字说明请求为什么需要分流。

表 1　LLaMA-70B 在三类 GPU 上的性能剖析数据（每层）

GPU	最大层数	Prefill（1000 token）	Decode（批量 2～12）
A100	12	8 ms	1.4 ms
L4	7	38 ms	6.9～7.1 ms
T4	4	59 ms	7.1～7.7 ms

2.2　多路径执行中的汇聚与分流

现有异构推理系统或通过放置与并行策略决定各设备提供多少容量[10-12]，或在给定部署上为请求选择执行路径[5,13]，优化目标主要是吞吐或单请求延迟。在多副本流水线中，各请求的执行还通过 Decode 批处理、路径的持续占用与链路共享相互耦合。为分离这些耦合，我们固定部署、调度器与到达轨迹，每次只改变一个因素，即可用路径集合、路由代价或某类激活的传输量（图 1）。以下三个观察分别对应引言中的三个环节。

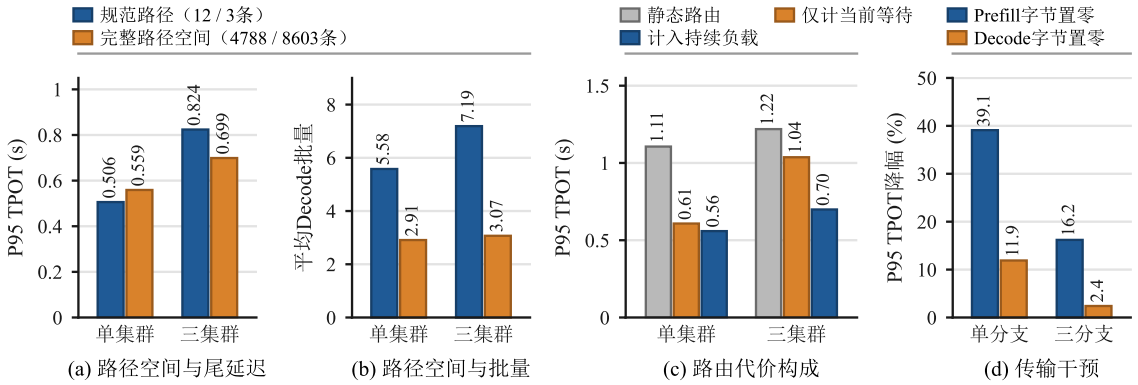


图 1　动机实验（固定部署、调度器与到达轨迹；单集群 700 token/s，三集群 320 token/s）。(a)(b) 仅改变可用路径集合，图例括号内为两种拓扑下的路径数；(c) 仅改变路由代价；(d) 在三集群拓扑上分别将节点间 Prefill 或 Decode 激活的传输字节数置零

观察 1：额外路径的利弊取决于分流位置。仅改变可用路径集合（图 1(a)(b)）。单集群部署的完整执行图有 4788 条路径，而规范路径（只沿副本链执行，见第 3.3 节）仅 12 条。开放全部路径后，单集群实际使用的路径增至约 507 条，平均 Decode 批量由 5.58 降至 2.91，P95 TPOT 上升 10.5%。三集群中情况相反：只走规范路径虽然保持了大批量，却使 Prefill 流量集中于少数慢链路，开放路径后 P95 TPOT 下降 15.2%。额外路径只有在绕开瓶颈链路时才有净收益，其余情况下主要是在分散请求；而 Helix[5]与 Parallax[13]对全部合法的跨副本连接同等对待。

观察 2：瞬时队列不能反映请求的持续负载。仅改变路由代价（图 1(c)）。只计当前等待的路由虽优于静态路由，三集群的 P95 TPOT 仍达 1.04 s；再计入路径上活跃请求的持续负载后降至 0.70 s，降低 32.6%，单集群也降低 8.1%。原因在于请求会在数百个 Decode 步中持续占用其路径，而任一时刻只有其中一步出现在队列中。经典负载均衡[27]、Helix 的最大流分配与 Parallax 的逐层最小延迟选路都没有计入这部分负载。

观察 3：长 Prefill 传输在慢链路上造成队头阻塞。分别将节点间 Prefill 或 Decode 激活的传输字节数置零（图 1(d)）。在单分支部署中，去掉 Prefill 传输使 P95 TPOT 降低 39.1%，去掉 Decode 传输只降低 11.9%，且慢链路上 99.7%以上的 Decode 等待与 Prefill 传输在时间上重叠；三分支部

署通过分流将两项降幅减小到 16.2%与 2.4%。可见长 Prefill 激活一旦占用链路与网卡，即使调度器优先发送 Decode，此后就绪的 Decode 也只能等待，形成队头阻塞[28,29]。计算侧的分块预填充[16]无法消除这种链路占用。

设计原则。三个观察都源于汇聚与分流的矛盾。不加选择地开放路径会打散 Decode 批处理，完全收缩路径又会放大慢链路上的排队与阻塞。RCAPS 据此在三个时间点控制路径空间，形成三条原则。（1）部署时共享前段、按需复制后段，使请求尽量汇聚以维持 Decode 批量（第 3.2 节）。（2）选路是请求生命周期中唯一的路径决策点，应在此为请求的持续负载计价（第 3.4 节）。（3）空间上只在慢链路附近开放绕行，时间上缩短长 Prefill 传输对链路的连续占用（第 3.3 节与第 3.5 节）。

3 方法

本节首先定义 RCAPS 需要做出的决策并给出总体框架（第 3.1 节），随后按部署、选路与传输的顺序介绍分段共享与复制放置（第 3.2 节）、路径空间裁剪（第 3.3 节）、持续负载感知选路（第 3.4 节）与 Prefill 传输分块（第 3.5 节），最后汇总完整算法（第 3.6 节）。

3.1 问题定义与总体框架

图 2 给出 RCAPS 的总体框架。设模型由L层组成，部署在 GPU 集合V上，物理连接(i,j)的带宽为B_{ij}。RCAPS 需要做出三类决策：服务启动前确定部署x（哪些 GPU 驻留哪些层）以及开放给在线路由的边集$\mathcal{E}_{adm}$；每个请求r到达时确定其执行路径p_r；慢链路传输时确定何时让出链路τ。这些决策满足

$$x \in \mathcal{X}, \quad \mathcal{E}_{\text{can}}(x) \subseteq \mathcal{E}_{\text{adm}} \subseteq \mathcal{E}_{\text{full}}(x), \quad p_r \in \mathcal{P}(\mathcal{E}_{\text{adm}}), \tag{1}$$

其中X为可行部署集合，$\mathcal{E}_{full}(x)$与$\mathcal{E}_{can}(x)$分别为部署导出的完整执行边集与规范执行边集，$P(\mathcal{E})$为边集$\mathcal{E}$诱导的完整路径集合。以$TTFT_r$与$TPOT_r$分别表示请求的首 token 延迟与平均输出 token 间隔，Q_{95}表示 95 分位数，优化目标为

$$\begin{aligned} \min_{x,\mathcal{E}_{\text{adm}},\{p_r\},\tau} \quad & (Q_{95}[\text{TPOT}_r],\ Q_{95}[\text{TTFT}_r]) \\ \text{s.t.} \quad & \text{式(1)、KV 容量与带宽约束.} \end{aligned} \tag{2}$$

式(2)中三类决策的时间尺度相差数个数量级：部署在服务启动时确定一次，选路随请求到达逐个进行，让出链路发生在单次传输内部，而且部署时无法预知未来的请求到达。RCAPS 因此不直接求解式(2)，而是将其分解为共享同一张执行图的四个模块（图 2）。放置器在服务启动时选择分段共享与复制放置，使请求在模型前段汇聚（第 3.2 节）；裁剪器在执行图上只为慢链路开放绕行，得到可用边集（第 3.3 节）；服务阶段，路由器在每个请求到达时以计入持续负载的路径代价选路（第 3.4 节）；执行器沿绑定路径执行，并将慢链路上的长 Prefill 激活分块发送（第 3.5 节）。这一分解不保证式(2)的全局最优，但使每个模块都能在各自的时间尺度内以较低开销完成决策。

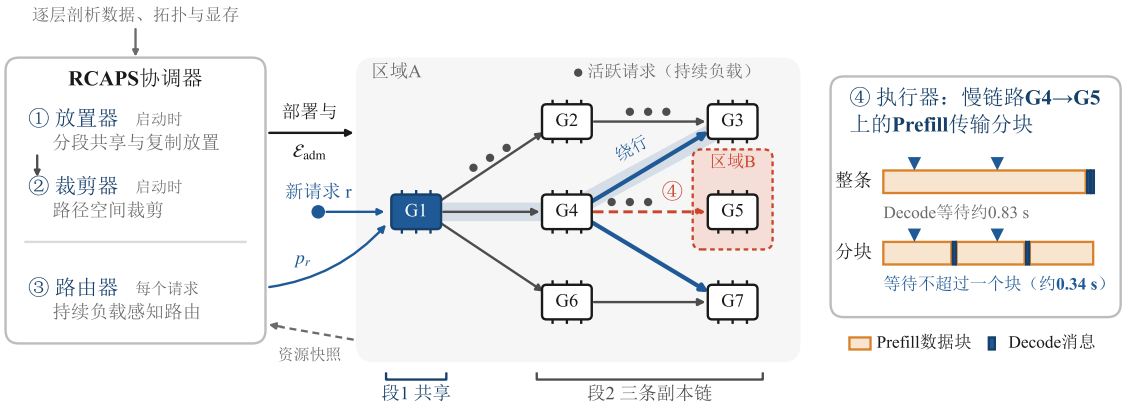


图 2　RCAPS 总体框架（以图 3、图 4 的部署为例）。①放置器与②裁剪器在服务启动时运行一次，确定部署与可用边集；③路由器在每个请求到达时，依据资源快照中的当前等待与活跃请求为其选路；④执行器沿绑定路径执行，并对慢链路上的 Prefill 激活分块发送

一个请求的生命周期。以三集群部署为例。服务启动时，放置器得到 1→3 结构，第一段由单一副本链承担，后续层段由三条副本链承担；裁剪器在此基础上只在跨地域慢链路的起点与终点附近开放高速绕行。请求到达时，路由器读取当前资源快照，计算每条可用边的等待、计算、传输与持续负载代价，在执行图上求出代价最小的路径并将其绑定到该请求。随后请求在第一段与其他所有请求一起组批，在后续层段沿绑定的副本链执行；若路径经过慢链路，其 Prefill 激活按 4 MiB 分块发送，块与块之间先发送已就绪的 Decode 消息。Prefill 完成后，该请求的每个 Decode 步都沿同一路径执行，并计入后续请求选路时看到的持续负载，直到输出结束、KV 缓存释放。

3.2　分段共享与复制放置

放置决定各层段由几个副本承担、各副本由哪些 GPU 组成。为使放置、裁剪与路由对合法路径采用同一定义，本文沿用 Helix[5]的层进度规则构造执行图。设 GPU i 驻留层区间$[a_i, b_i)$，请求在 GPU i 上执行到层进度边界b_i后，可以转入任何驻留第b_i层及其后所需各层的 GPU j，即

$$(i,j) \in \mathcal{E}_{\text{full}}(x) \Leftrightarrow (i,j) \in \mathcal{E}_{\text{phy}},\ a_j \le b_i < b_j. \tag{3}$$

两台 GPU 的层区间只需相接或重叠，不必首尾对齐。本文以从 0 开始的半开区间[a,b)表示驻留的层索引，右端点 b 即执行完成后的层进度边界。如图 3 所示，G4 驻留[6,12)，即执行索引 6～11 的层，G3 驻留[11,16)；请求在 G4 到达层进度边界 12 后，由 G3 从索引 12 的层继续执行，不重复执行索引 11 的层。每次转移都严格推进已完成的层边界，因此执行图无环，任一完整路径恰好覆盖全部层。

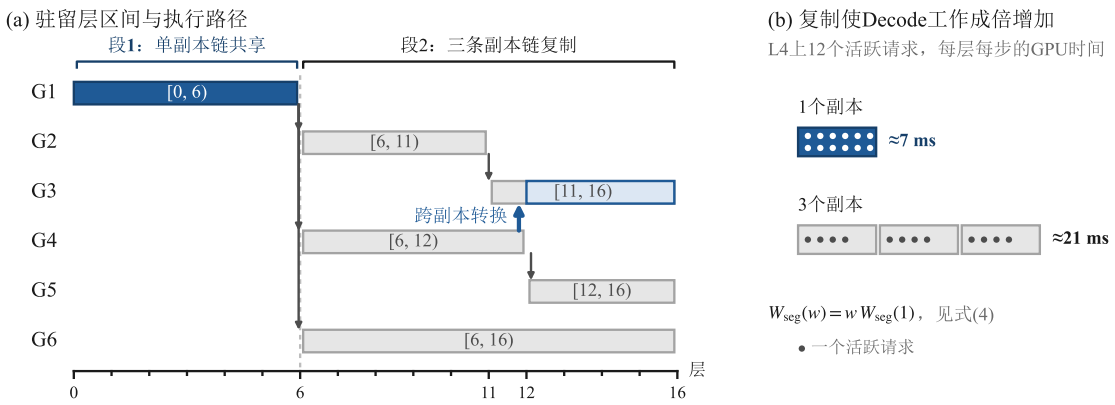


图 3　分段共享与复制放置示例（模型共 16 层）。(a) 各 GPU 的驻留层区间：段 1 由 G1 单独承担并被所有请求共享，段 2 由三条副本链复制；G4 到达层进度边界 12 后可由驻留[11, 16)的 G3 继续执行（满足式(3)），浅蓝部分为 G3 此时实际执行的区间。(b) 以 L4 为例，12 个活跃请求集中在一个副本与分到三个副本时每层每步的 GPU 时间

复制层段可以为 Prefill 计算与跨节点传输提供并行通道，但会将 Decode 请求分散到更多副本上。由于 Decode 单步耗时在小批量范围内基本不变（表 1），设单步耗时为g，N个活跃请求均匀分布在w个副本上，则每生成一个 token，该层段的 GPU 工作量为

$$W_{\mathrm{seg}}(w) = \frac{w\,g}{N} = w\,W_{\mathrm{seg}}(1). \tag{4}$$

即复制w份会使该层段的 Decode 工作放大w倍，引言中 L4 上 7 ms 与 21 ms 的对比就是这一关系的实例。因此 RCAPS 采用先汇聚、后分流的结构。第一段由单一副本链承担，所有请求都在此汇聚组批；后续各段只在瓶颈处复制，为 Prefill 计算与慢链路传输提供分流通道。单集群的搜索结果为 1→4→3 结构，三集群为 1→3 结构。

部署时无法直接计算尾延迟，放置器因此用三个可离线计算的代理量评价候选放置，即规范路径上最慢的 Prefill 时间$T_P(x)$、最慢的 Decode 时间$T_D(x)$，以及均匀分流时负载最重的资源需求$D_{max}(x)$。以参考放置x_{ref}归一化后，目标为

$$\begin{aligned} J(x) = \quad & \lambda_{\mathrm{P}}\frac{T_{\mathrm{P}}(x)}{T_{\mathrm{P}}(x^{\mathrm{ref}})} + \lambda_{\mathrm{D}}\frac{T_{\mathrm{D}}(x)}{T_{\mathrm{D}}(x^{\mathrm{ref}})} \\ & +\lambda_{\mathrm{B}}\frac{D_{\max}(x)}{D_{\max}(x^{\mathrm{ref}})}, \end{aligned} \tag{5}$$

其中权重默认取$(\lambda_P, \lambda_D, \lambda_B) = (0.40, 0.35, 0.25)$。代理目标只用于对候选放置排序，不用于预测延迟。搜索时，分段边界以 4 层为步长取值，后续各段的副本数取自{1,2,3,4}；对每种分段形状，用宽度为 64 的束搜索将 GPU 分组到各副本，并按设备性能分配层数，最后选出$J(x)$最小的放置。整个搜索只在服务启动时运行一次（开销见第 4.4 节）。

3.3　保留规范结构的路径空间裁剪

观察 1 表明，额外的跨副本路径只在绕开慢链路时才有净收益，其他情况下主要是在分散请求。裁剪器因此保留放置给出的全部规范连接$\mathcal{E}_{can}$（请求沿副本链执行，只在段边界切换副本），只在慢链路附近开放绕行。给定带宽阈值θ_S，慢链路集合为

$$\mathcal{E}_{\mathrm{slow}} = \{(i,j) \in \mathcal{E}_{\mathrm{can}} \colon B_{ij} \le \theta_S\}, \tag{6}$$

记慢链路的起点集合与终点集合分别为$\mathcal{A}$与$\mathcal{B}$，则额外开放的边与最终的可用边集为

$$\begin{aligned} \mathcal{E}_{\mathrm{ext}} = \{(i,j) \in \quad & \mathcal{E}_{\mathrm{full}} \backslash \mathcal{E}_{\mathrm{can}} \colon B_{ij} > \theta_S, \\ & i \in \mathcal{A} \ \vee \ j \in \mathcal{B}\}, \end{aligned} \tag{7}$$

$$\mathcal{E}_{\mathrm{adm}} = \mathcal{E}_{\mathrm{can}} \cup \mathcal{E}_{\mathrm{ext}}. \tag{8}$$

本文取$\theta_S = 50$ MB/s。如图 4 所示，裁剪器只在慢链路 G4→G5 的起点开放两条高速绕行 G4→G3 与 G4→G7，可用的完整路径由 3 条增至 5 条，完整执行图中的其余 4 条路径不开放。部署中没有低于阈值的链路时，可用边集退化为规范边集。

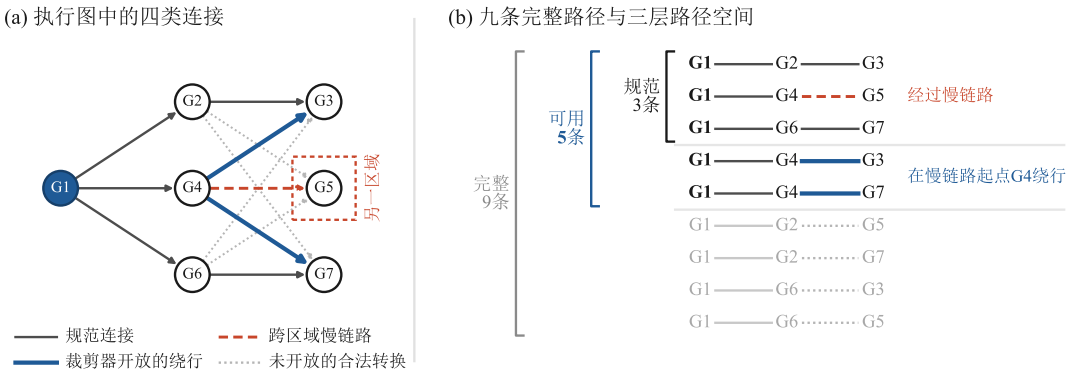


图 4　同一部署上的三种路径空间。(a) 执行图中的四类连接，G5 位于另一区域，G4→G5 为跨区域慢链路；(b) 九条完整路径，规范路径空间（3 条）、裁剪后的可用路径空间（5 条）与完整执行图（9 条）依次包含

3.4　持续负载感知的层序动态规划选路

观察 2 表明，只看当前队列会低估已有请求的负担。在固定路径执行模型下，路径一经绑定，请求就会在数百个 Decode 步中反复占用沿途 GPU，而任一时刻只有其中一步出现在队列中。因此，路由器除当前等待外，还以单请求 Decode 的剖析时间为活跃请求的持续负载计价，称为持续负载代价：

$$C_j^{\mathrm{D}}(h) = \kappa \sum_{\ell=h}^{b_j-1} f_{j,\ell}^{\mathrm{D}}(1), \tag{9}$$

其中$\kappa = 2$用于使剖析时间与执行器的计时口径一致。记$N_j(t)$为经过 GPU j的活跃请求数，$W_j^G(t)$与$W_{ij}^N(t)$为资源快照给出的 GPU 与连接当前等待，C_j^P为新请求的 Prefill 计算时间，$\tau_{ij}(r)$为传输时间，则请求由 GPU i转入 GPU j的代价为

$$\begin{aligned} c_{ij}(r,t) = \;& C_j^{\mathrm{P}}(b_i, r) + W_j^{\mathrm{G}}(t) + N_j(t)\, C_j^{\mathrm{D}}(b_i) \\ & + W_{ij}^{\mathrm{N}}(t) + \tau_{ij}(r), \end{aligned} \tag{10}$$

式(10)各项均以秒计，沿路径可加。其中持续负载项使已承载较多活跃请求的 GPU 在比较中代价更高，从而避免新请求持续涌向同一路径，它并不预测新请求的实际延迟。这种分散与第 3.2 节的汇聚可以共存。第一段只有一个副本，所有请求无论选择哪条路径都在此组批，持续负载代价只改变请求在复制后段上的分布。由于各项可加，选路转化为可用执行图上的最短路问题。请求进入一个 GPU 后总是执行到其驻留终点，每个 GPU 只对应一个动态规划状态，按驻留终点递增的顺序即可递推最小累计代价：

$$F_r(b_j, j) = \min_{\substack{(i,j)\in\mathcal{E}_{\mathrm{adm}} \\ a_j \le b_i < b_j}} \{F_r(b_i, i) + c_{ij}(r,t)\}. \tag{11}$$

完成全部层后回溯，即得到给定资源快照与代价模型下代价最小的路径。执行图按层进度构成有向无环图，一次动态规划只需遍历各顶点并对每条可用边松弛一次，时间复杂度为 O(|V|+|E|)，其中|E|为可用边数，与完整路径的数目无关。以单集群部署为例，完整执行图有 4788 条路径，路由器只需处理 62 条可用边。

3.5 Prefill 传输分块

观察 3 表明，慢链路上的一次 Prefill 激活传输可持续秒级，其间就绪的 Decode 只能等待。执行器因此对带宽不超过θ_T的 GPU 间连接，将 Prefill 激活按块长q切分发送，每个块发送结束后先发送已就绪的 Decode 消息，再继续下一块。下游 GPU 仍在收到完整激活后才开始计算，分块只改变链路占用的时间顺序，不改变计算粒度。设第h个块的大小为$M_h \le q$，获配带宽为v，其连续占用时间满足

$$t_h^{\mathrm{ser}} = \frac{M_h}{v} \le \frac{q}{v}. \tag{12}$$

本文取$\theta_T = 50$ MB/s、$q = 4$ MiB。如图 5 所示，在 12.5 MB/s 的跨集群链路上，750 个 token 的 Prefill 激活约 12.3 MB，整条发送时 Decode 最多等待约 1 s，分块后不超过约 0.34 s。这一上界要求块边界是发送方可以重新调度的位置；在真实 TCP 协议栈中，还需要限制已交给内核但尚未发出的数据量（第 4.4 节）。

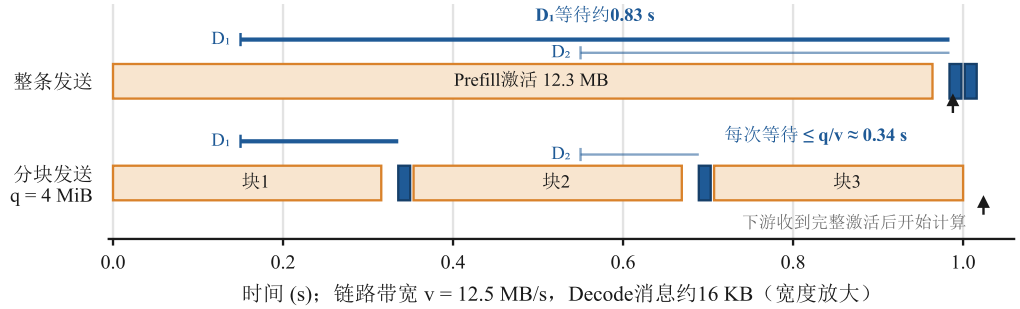


图 5　慢链路（12.5 MB/s）上 Prefill 激活传输的链路占用时序。蓝色横线为 Decode 消息从就绪到发出的等待区间：整条发送时 D1 需等待约 0.83 s，按 4 MiB 分块后每次等待不超过一个块的占用时间；数据条下方的黑色小箭头为下游收到完整激活、开始计算的时刻

3.6 算法实现流程

算法 1 汇总 RCAPS 的完整流程。离线阶段在服务启动时运行一次，完成放置与路径空间裁剪；在线阶段对每个到达的请求选路并绑定路径，执行时对慢链路上的 Prefill 激活分块发送。

算法 1　RCAPS 算法实现流程

输入：逐层性能剖析数据，物理拓扑与显存容量，阈值θ_S、θ_T，块长q
输出：部署x，可用边集$\mathcal{E}_{adm}$，各请求的执行路径p_r
离线阶段：放置与路径空间裁剪
枚举分段形状（第一段固定为单副本共享段，$K \in \{2,3\}$）
for 每种分段形状 **do**
　　用束搜索将 GPU 分组到各副本，并按设备性能分配模型层数，得到候选放置
end for
由式(5)评价全部候选放置，选出$J(x)$最小的部署x
由式(3)构造执行图，由式(6)～(8)确定可用边集$\mathcal{E}_{adm}$
在线阶段：持续负载感知选路与分块传输
for 每个到达的请求r **do**
　　读取资源快照，计算当前等待与各 GPU 的活跃请求数$N_i(t)$
　　按驻留终点递增的顺序，由式(10)与式(11)计算各 GPU 的最小累计代价
　　选出完成全部层的终端 GPU，回溯得到路径p_r，将其绑定到请求r
　　沿p_r执行 Prefill 与后续 Decode；对带宽不超过θ_T的连接，按块长q分块发送 Prefill 激活，并在块边界优先发送已就绪的 Decode
end for

4 实验评估

本节回答三个问题：RCAPS 相对现有方法能否降低尾延迟（第 4.2 节）；放置、路由、裁剪与分块各自贡献多少，收益从何而来（第 4.3 节）；这些结论在网络条件与建模误差变化时是否成立，在什么条件下失效（第 4.4 节）。

4.1 实现与实验设置

实现与平台。我们在 Helix 开源的离散事件集群模拟器[5]上实现 RCAPS 的完整原型。模拟器以逐层性能剖析数据驱动 GPU 计算，并对链路带宽、传播时延与收发网卡的共享占用建模；一次传输开始时获得一份带宽额度并独占至发送结束，对应节点之间通过持久连接依次发送消息的常见实现。本地 Helix 在三集群 320 token/s 下的输出与官方工件保存的参考结果逐项一致。所有方法共享同一模拟器、剖析数据与请求轨迹，差异只来自放置、路由与执行策略。RCAPS 的全部变体使用相同的执行规则，即 Decode 优先发送并合并同一跳的消息，GPU 组批时优先纳入 Decode。

集群与负载。两种拓扑都由 24 张异构 GPU 组成（4 张 A100、8 张 L4、12 张 T4）。单集群中节点之间为GB/s级高速互联；三集群将同样的GPU分布到三个地域，区域内平均带宽 1.25 GB/s、时延 1 ms，跨区域平均带宽 12.5 MB/s、时延 50 ms。主实验使用 LLaMA-70B。请求的输入、输出长度与到达过程取自 Azure LLM 对话轨迹[18]，并按目标 token 吞吐缩放到达强度。我们以所有方法都能完整运行的最高负载（单集群 350 token/s、三集群 160 token/s）作为共同负载点，以单集群 700 token/s、三集群 320 token/s 作为高负载点。

对比方法。外部基线包括：（1）Helix[5]，使用官方开源实现的放置、最大流路由、在线参数与执行方式；（2）Helix-tuned，在不参与评估的随机种子上对 Helix 的执行开关与在线参数调优后得到的最强配置，其调优空间包含 RCAPS 所用的 Decode 优先与合并发送规则，用于排除基线未充分调优的解释；（3）Swarm[9]与（4）独立流水线（SP），使用 Helix 中的部署与调度器。此外，我们构造控制组 Helix+，保留 Helix 的 MILP 放置与最大流路由，只将执行规则替换为与 RCAPS 相同的规则。Helix+不作为外部基线，只用于在相同执行规则下分离放置与路由的贡献。对每个负载点和每项指标，本文将 Helix 与 Helix-tuned 中较优者称为最强 Helix 基线。三集群上 Helix 部署使用 24 张 GPU、122 个层副本，RCAPS 使用 23 张 GPU、144 个层副本，二者满足相同的逐卡显存约束。

指标与协议。主要指标为请求级 P95 TPOT 与 P95 TTFT，同时报告窗口内输出吞吐、完成率与 GPU 秒/token（全部 GPU 忙碌时间之和除以输出 token 数）。主实验采用 1800 s 固定窗口，另用预热 300 s 后排空全部请求的协议复核。每个配置运行 5 个随机种子，同一种子下各方法的请求序列相同；图表中的数值均为 5 个种子的均值，误差棒为标准差，相对变化按种子配对计算。RCAPS 的参数固定为$\theta_S = \theta_T = 50$ MB/s、$q = 4$ MiB、$\kappa = 2$。

4.2 端到端服务性能

与外部方法的比较。表 2 比较 RCAPS 与 Helix、Helix-tuned 的端到端结果。在共同负载下，RCAPS 相对最强 Helix 基线将 P95 TPOT 降低 51.6%（单集群）与 65.6%（三集群），相对 Swarm 与独立流水线降低 63.5%～79.8%；在高负载下，降幅扩大到 64.5%与 81.6%，三集群的完成率比最强 Helix 基线高 22.5 个百分点。P95 TTFT 的变化方向一致，在共同负载下降低 40.1%与 61.4%，在高负载下降低 44.3%与 66.1%。在相同到达负载下，RCAPS 的窗口内吞吐均不低于各 Helix 基线，三集群高负载下提高 22.3%。收益来自两方面。分段放置使所有请求在共享前段汇聚，后段复制又为慢链路提供分流通道；持续负载代价使新请求避开负担已重的路径（第 4.3 节）。网络越

不均匀、负载越高，排队与阻塞越严重，RCAPS 的优势也越大。改用预热并排空的协议后结论不变，说明优势并非来自固定窗口对未完成请求的截断。

表 2　端到端结果（TPOT 与 TTFT 为 P95 值，单位 s；吞吐单位 token/s；GPU 秒指 GPU 秒/token；完成率单位%；↓表示越低越好，↑表示越高越好，加粗为同一负载下各列最优。单集群中 Helix-tuned 选出的配置即控制组 Helix+；Swarm 与 SP 劣于各 Helix 基线，正文给出其相对值）

负载	方法	单集群（共同 350 / 高 700 token/s）					三集群（共同 160 / 高 320 token/s）				
		TPOT↓	TTFT↓	吞吐↑	GPU 秒↓	完成率↑	TPOT↓	TTFT↓	吞吐↑	GPU 秒↓	完成率↑
共同负载	RCAPS	**0.397**	**3.28**	**77.48**	0.1195	**95.1**	**0.527**	**5.11**	**35.76**	**0.1137**	**93.9**
	Helix-tuned	0.820	5.48	75.12	**0.0605**	91.6	1.536	13.56	33.25	0.2349	85.4
	Helix	1.084	5.63	73.42	0.1498	88.7	1.556	13.25	33.20	0.2329	84.3
高负载	RCAPS	**0.508**	**3.61**	**156.88**	0.0530	**93.8**	**0.612**	**5.27**	**71.52**	**0.0642**	**92.2**
	Helix-tuned	1.428	6.48	146.19	**0.0275**	85.6	3.350	15.55	58.47	0.0887	69.7
	Helix	1.734	6.67	141.75	0.0578	81.8	3.447	15.78	58.14	0.0871	68.8

执行规则与调优不能弥补差距。为排除收益来自执行细节的可能，控制组 Helix+将 RCAPS 的执行规则移植给 Helix。单集群中，Helix+的 P95 TPOT 比 Helix 降低 17.6%～24.3%，Helix-tuned 在该拓扑上选出的也正是这一配置；三集群中，Helix+的 P95 TPOT 反而上升 5.1%～19.4%，因为最大流路由集中分配的请求在跨区域路径上排队更长。在独立种子上进一步调优后，三集群的 Helix-tuned 也只比 Helix+改善 6.1%～18.5%；在两种拓扑上，最强 Helix 基线与 RCAPS 之间仍有 51.6%～81.6%的差距。可见 RCAPS 的收益主要来自放置与路由。

负载变化。图 6 给出不同负载下的尾延迟与完成率。在相同执行规则下，负载从最低升到最高时，Helix+的 P95 TPOT 在单集群与三集群分别增至原来的 2.7 倍与 5.2 倍，RCAPS 只增至 1.5 倍与 1.3 倍。Helix 的最大流路由按稳态流量比例分配请求，不感知各路径上已经累积的持续负载，负载升高时请求持续涌向少数拥塞路径；RCAPS 则在每次选路时将这部分负载计入代价。外部方法也先于 RCAPS 达到承载边界，而 RCAPS 在全部测试负载下都能完成运行。

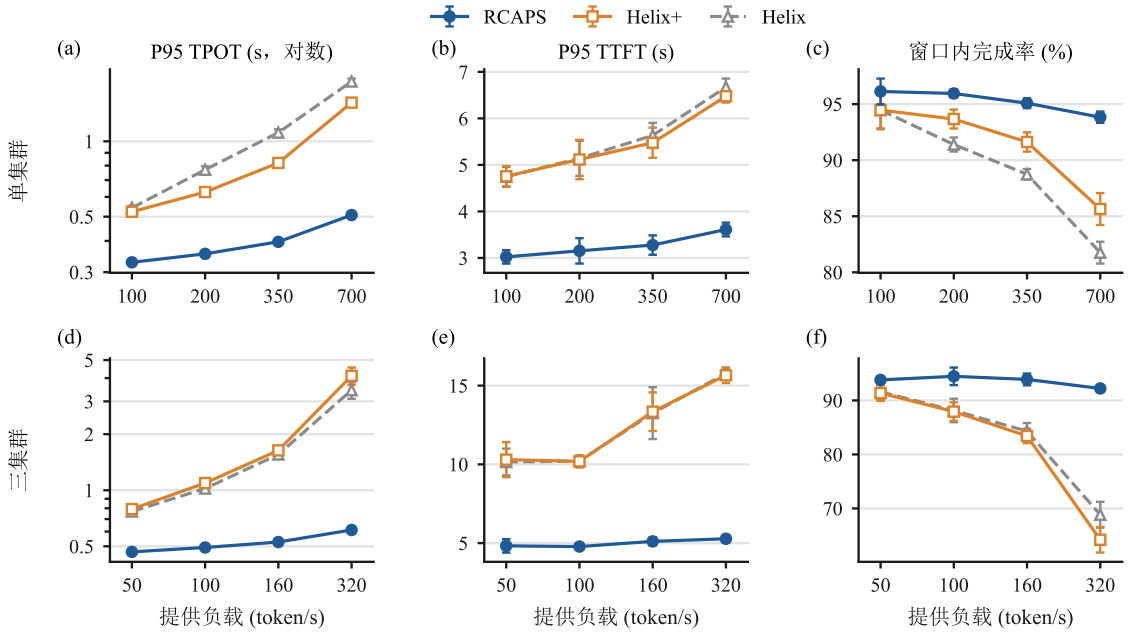


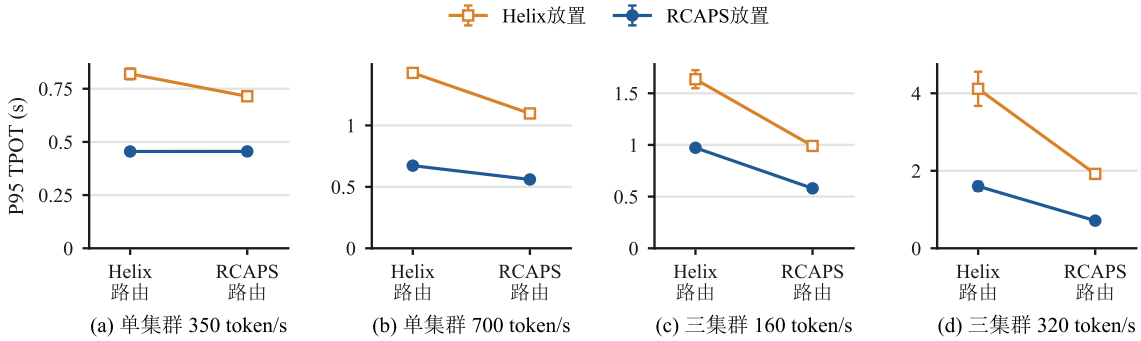
图 6　不同负载下的 P95 TPOT、P95 TTFT 与窗口内完成率。(a)～(c) 单集群，(d)～(f) 三集群；TPOT 为对数纵轴，误差棒为 5 个种子的标准差

计算代价与有效产出。相对执行规则相同的 Helix+，RCAPS 以更多的计算换取更低的尾延迟，其 GPU 秒/token 高 93%～161%，单集群中这也是相对 Helix-tuned 的差距。三集群中，Helix-tuned

为压低尾延迟选出的执行配置计算量更大，RCAPS 的 GPU 秒/token 反而比它低 28%～52%。RCAPS 的 GPU 秒/token 也低于 Helix、Swarm 与独立流水线。额外计算主要来自持续负载代价对请求的分散（第 4.3 节）。按满足 SLO 的有效产出衡量，这一取舍是合算的。在中等 SLO（单集群 TTFT≤6 s 且 TPOT≤0.6 s，三集群 TTFT≤10 s 且 TPOT≤0.8 s）下，RCAPS 的 goodput 为最强 Helix 基线的 2.5～49.5 倍，按 GPU 忙碌秒归一化后仍为 Helix+的 1.2～26 倍。这一取舍也有不划算的区间。负载较低且 SLO 足够宽松时，两者 goodput 相同，而 Helix+只需约一半的计算；此时可改用第 4.3 节中按典型批量摊薄的持续负载代价，或改用更紧凑的 1→2 放置，在单集群中以 P95 TPOT 升高 20.8%为代价减少 62%的计算。

4.3 消融实验与机制分析

放置是主要收益来源。图 7 在统一执行规则下交叉组合两种放置与两种路由，并关闭路径空间裁剪与传输分块。将 Helix 放置换成 RCAPS 放置后，无论使用哪种路由，P95 TPOT 都降低 36%～63%；在 RCAPS 放置上更换路由，单集群 350 token/s 下没有可分辨的差别，只在高负载或三集群中带来 17%～55%的收益。两者的收益呈次可加性，好的放置已经缓解了主要的资源竞争，路由能改进的余量随之减小。



(a) 单集群 350 token/s　(b) 单集群 700 token/s　(c) 三集群 160 token/s　(d) 三集群 320 token/s

图 7　统一执行规则下放置与路由的交叉对比（关闭路径空间裁剪与 Prefill 传输分块）

收益来自共享前段这一结构约束。为确认收益并非任何更好的放置都能带来，我们保持路由、裁剪、分块与执行规则不变，只替换放置（表 3 上部）。使用同一目标函数、同一搜索算法与预算，仅去掉第一段必须由单一副本链共享的约束，P95 TPOT 在单集群 700 token/s 与三集群 320 token/s 下分别升高 39.1%与 73.5%；同预算随机搜索与贪心放置同样更差。可见仅靠目标函数与搜索得不到这一结构，共享前段约束起了决定作用。另一方面，三种替代放置的 GPU 秒/token 为 0.0144～0.0320，均低于默认放置。它们的 Decode 批量更大、计算更省，代价是更长的排队，这与 RCAPS 以计算换延迟的取舍一致。

表 3　放置与持续负载代价的消融（每行只替换所列组件，其余与完整 RCAPS 相同）

类别	变体	单集群 700 token/s		三集群 320 token/s	
		P95 TPOT/s↓	GPU 秒/tok↓	P95 TPOT/s↓	GPU 秒/tok↓
	完整 RCAPS（默认）	**0.508**	0.0530	**0.612**	0.0642
放置替换	去掉共享前段约束	0.706	0.0179	1.062	0.0174
	同预算随机搜索	0.691	0.0173	0.813	0.0320
	贪心放置	0.762	0.0160	1.091	0.0144
路由替换	不计持续负载代价	0.645	0.0363	0.828	0.0373
	边际代价	0.625	0.0362	0.829	0.0364
	按典型批量摊薄	0.501	0.0529	0.664	0.0527
参考	控制组 Helix+	1.428	0.0275	4.116	0.0246

持续负载代价是路由有效的关键。相对只去掉持续负载代价、保留当前等待的版本，RCAPS 在单集群 700 token/s 与三集群 320 token/s 下使 P95 TPOT 降低 21.3%与 26.1%（表 3 下部）。不计持续负载代价时，路由器只看到当前队列，三集群中 78%的请求涌向同一条路径，平均 Decode 批量高达 40.5，计算量很低，但每输出 token 的网络等待是 RCAPS 的 2.3 倍；计入持续负载代价后，请求分散到 8 条路径，这也是 GPU 秒/token 增加 46%～72%的原因。按典型批量摊薄的持续负载代价在单集群与默认版本相当（P95 TPOT 为 0.501 s 与 0.508 s），在三集群 P95 TPOT 略高（0.664 s 与 0.612 s）而 GPU 秒/token 更低（0.0527 与 0.0642），适合更看重计算效率的场景。在相同放置与执行器下，RCAPS 路由相对 Parallax 式逐层最小延迟路由[13]使 P95 TPOT 降低 9.2%～36.9%，说明这部分收益主要来自持续负载代价项，动态规划只是求解手段。

裁剪与分块互补。图 8 在三集群 320 token/s 下组合路径空间裁剪与传输分块。只加裁剪时，请求使用的路径由平均 33.4 条收敛到 8 条，批量增大，GPU 秒/token 降低 30%，但慢链路上的排队略有增加，P95 TPOT 上升 1.5%。再加上传输分块后，长 Prefill 传输不再连续占用慢链路，P95 TPOT 比两者都不加时低 14.1%，GPU 秒/token 仍低 28%。只保留规范路径虽然计算最省，P95 TPOT 却高 9.8%，说明在慢链路附近保留绕行是必要的。裁剪从空间上限制分流位置，分块从时间上缓解路径收敛后慢链路的拥挤，两者需要配合使用。

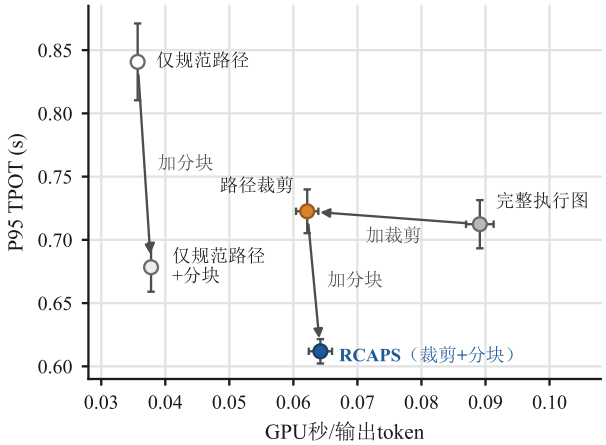


图 8　三集群 320 token/s 下路径空间裁剪与传输分块的组合

4.4　敏感性、适用边界与系统开销

队头阻塞在真实 TCP 协议栈中同样存在。分块的收益以链路上存在队头阻塞为前提，将模拟器的链路模型改为流公平共享后，分块不再起作用。为检验真实协议栈的行为，我们在 WSL2 的 Linux 内核中用 netem 搭建限速虚拟链路，在同一条 TCP 连接上发送 Prefill 激活与 Decode 消息（表

4）。不分块时 Decode 消息的 P95 时延达 2.41 s；4 MiB 与 1 MiB 分块分别将其降至 0.83 s 与 0.23 s，大消息的完成时间只增加 13%～18%。若为 Decode 单独建立连接，时延可降至 0.03 s，此时分块不再必要。因此，分块适用于单连接顺序发送的实现；该实验检验的是传输机制本身，不代表完整推理系统的端到端表现。

表 4　Linux TCP 协议栈上的传输分块效果（netem 限速虚拟链路，带宽单位 MB/s，10 次重复的均值；前三行为单连接顺序发送，大消息完成时间在 12.5 MB/s 下测得）

发送方式	Decode P95 时延/s↓			大消息完成/s↓
	12.5	25	50	
单连接不分块	2.41	2.18	2.06	2.82
4 MiB 分块	0.83	0.74	0.70	3.19
1 MiB 分块	0.23	0.21	0.20	3.32
大小消息分连接	0.030	0.027	0.026	2.87

跨区域带宽。跨区域带宽为 12.5 MB/s 与 25 MB/s 时，分块使 P95 TPOT 降低 15.3%与 9.2%；带宽达到 50 MB/s 及以上时没有链路低于阈值，裁剪与分块自动退化，结果与关闭两项机制时相同。

建模误差与适用边界。对计算剖析、带宽、时延、激活与 KV 大小施加扰动后，RCAPS 相对 Helix+的 P95 TPOT 降幅在单集群保持在 63.6%～64.9%，在三集群保持在 80.6%～85.6%。本文的评估有三点边界。第一，全部端到端结果来自离散事件模拟，规模为 24 张 GPU，模型为 LLaMA-70B，尚未在真实异构集群与推理引擎上验证。第二，表 1 列出的 Decode 剖析范围为批量 2～12，而第 4.3 节中部分变体的平均批量超出这一范围，这些变体的计算时间取决于模拟器剖析数据在更大批量上的准确性。第三，输入长度放大 2 倍及以上时，所有方法都因 KV 容量不足或超出剖析范围而无法完成运行。

系统开销。放置搜索只在服务启动时运行一次，在 24 张 GPU 上约需 43 s（Intel Core i9-14900KF，Python 3.10 单线程）。在线选路的计算量与可用边数成线性，平均约 1.3 ms，相对秒级的 TTFT 可以忽略。各配置的 KV 缓存峰值占用不超过 38%。

5　相关工作

异构环境下的大模型推理。Petals[8]与 Swarm[9]在分布式、不可靠的异构设备上执行流水线推理与训练。面向推理服务，HexGen[10]、ThunderServe[11]、HexGen-2[30]与 Hetis[12]联合决定异构 GPU 上的并行策略与阶段部署，LLM-PQ[26]结合阶段感知切分与自适应量化，Mélange 等工作从成本角度选择 GPU 类型[6,31]，SpotServe 与 SkyServe 利用可抢占实例与跨地域资源降低成本[7,32]。Helix[5]以混合整数规划与最大流联合确定放置与请求分配，RCAPS 的层进度执行图沿用了它的连接规则。这些工作主要优化容量、吞吐与成本；RCAPS 关注多路径执行中汇聚与分流的取舍，并在部署、选路与传输三个时间点控制路径空间。

层图选路与负载均衡。Parallax[13]为每个请求在层索引图上选择逐层延迟最小的跨副本执行链。RCAPS 同样使用层序动态规划，区别有两点：路由运行在经过裁剪的可用边集上，路径代价计入了活跃请求的持续负载。在相同放置与执行器下，后者使 P95 TPOT 降低 9.2%～36.9%。经典的随机化负载均衡按到达时可观察的瞬时队列长度分派作业[27]；流水线中的请求则会在选定路径上驻留数百个 Decode 步，选路代价需要覆盖其整个生命周期。

请求调度与批处理。Orca[15]与 vLLM[14]分别改进了迭代级调度与 KV 缓存管理；FastServe[33]、Llumnix[34]、ExeGPT[35]、AlpaServe[36]、MuxServe[37]、NanoFlow[38]、

LoongServe[39]与 SGLang[40]从抢占、迁移、资源分配、复用与算子重叠等角度提升同构集群的服务效率。这些机制作用于单个 GPU 或单个服务实例内部，与 RCAPS 正交；RCAPS 面向异构设备与非均匀网络，在多路径流水线中决定批处理发生的位置与持续负载的分布。

阶段协同与通信优化。Sarathi-Serve[16]将长输入的 Prefill 计算分块并与 Decode 拼接执行；DistServe、Splitwise 与 TetriInfer[17-19] 将两个阶段分离部署；Mooncake、DéjàVu 与 CacheGen[20,41,42]优化 KV 缓存的存储与传输；MoLink[28]协调消费级 GPU 上的计算与通信。RCAPS 的传输分块作用于流水线节点之间的 Prefill 激活传输，只改变链路占用的时间顺序，不改变下游计算粒度，并与路径空间裁剪配合使用。网络领域早已认识到长消息对短消息的阻塞，pFabric[43]按剩余大小调度数据包，QUIC[29]以多流复用避免队头阻塞；RCAPS 在应用层以块为粒度实现类似的让出，适用于单连接顺序发送的流水线实现。

6 结论

本文研究异构 GPU 与非均匀网络上的多路径流水线推理。现有系统将可执行路径视为服务容量，本文的测量表明，在交互式服务中分流的价值随位置与时间而变。远离慢链路的分流会打散 Decode 批处理，请求对路径的占用贯穿整个 Decode 过程，长 Prefill 传输则会独占慢链路。RCAPS 据此将可执行路径空间作为受控资源，以分段共享与复制放置兼顾汇聚与分流，以路径空间裁剪限定额外分流的位置，以持续负载感知路由在选路时为请求的整个生命周期计价，以传输分块缩短长 Prefill 激活对慢链路的连续占用。在 24 张异构 GPU 上的模拟实验中，RCAPS 相对各负载点上该指标最优的 Helix 基线将 P95 TPOT 降低 51.6%～81.6%，将 P95 TTFT 降低 40.1%～66.1%，其中分段共享放置贡献了主要收益。这些收益以更多的 GPU 计算为代价，裁剪与分块也只在存在慢链路时生效。

后续工作将首先在真实异构 GPU 集群与推理引擎上部署 RCAPS，测量真实网络共享与块边界开销；其次补充更大 Decode 批量与长输入下的剖析数据，并引入 KV 感知的请求准入；再次设计可按 SLO 与资源预算在尾延迟与计算量之间折中的持续负载代价；最后探索在线调整裁剪阈值与块长的自适应机制。

参考文献

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[2] Hugo Touvron et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[3] Abhimanyu Dubey et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[4] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2): 74–80, 2013.

[5] Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang, Zhihao Jia, and Rashmi Vinayak. Helix: Serving large language models over heterogeneous GPUs and network via max-flow. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.

[6] Tyler Griggs et al. Mélange: Cost efficient large language model serving by exploiting GPU heterogeneity. *arXiv preprint arXiv:2404.14527*, 2024.

[7] Ziming Mao et al. SkyServe: Serving AI models across regions and clouds with spot instances. *arXiv preprint arXiv:2411.01438*, 2024.

[8] Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Max Ryabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. Petals: Collaborative inference and fine-tuning of large models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (System Demonstrations)*, 2023.

[9] Max Ryabinin, Tim Dettmers, Michael Diskin, and Alexander Borzunov. SWARM parallelism: Training large models can be surprisingly communication-efficient. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[10] Youhe Jiang, Ran Yan, Xiaozhe Yao, Yang Zhou, Beidi Chen, and Binhang Yuan. HexGen: Generative inference of large language model over heterogeneous environment. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[11] Youhe Jiang, Fangcheng Fu, Xiaozhe Yao, Taiyi Wang, Bin Cui, Ana Klimovic, and Eiko Yoneki. ThunderServe: High-performance and cost-efficient LLM serving in cloud environments. In *Proceedings of Machine Learning and Systems (MLSys)*, 2025.

[12] Zizhao Mo, Jianxiong Liao, Huanle Xu, et al. Hetis: Serving LLMs in heterogeneous GPU clusters with fine-grained and dynamic parallelism. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2025.

[13] Chris Tong, Youhe Jiang, Gufeng Chen, Tianyi Zhao, Sibian Lu, Wenjie Qu, Eric Yang, Lynn Ai, and Binhang Yuan. Parallax: Efficient LLM inference service over decentralized environment. *arXiv preprint arXiv:2509.26182*, 2025.

[14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[15] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.

[16] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

[17] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

[18] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *Proceedings of the 51st ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2024.

[19] Cunchen Hu et al. Inference without interference: Disaggregate LLM inference for mixed downstream workloads. *arXiv preprint arXiv:2401.11181*, 2024.

[20] Ruoyu Qin et al. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST)*, 2025.

[21] Yanping Huang et al. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[22] Deepak Narayanan et al. PipeDream: Generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*, 2019.

[23] Deepak Narayanan et al. Efficient large-scale language model training on GPU clusters using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.

[24] Lianmin Zheng et al. Alpa: Automating inter- and intra-operator parallelism for distributed deep learning. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.

[25] Reza Yazdani Aminabadi et al. DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2022.

[26] Juntao Zhao, Borui Wan, Yanghua Peng, Haibin Lin, and Chuan Wu. LLM-PQ: Serving LLM on heterogeneous clusters with phase-aware partition and adaptive quantization. In *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP), Poster*, 2024.

[27] Michael Mitzenmacher. The power of two choices in randomized load balancing. *IEEE Transactions on Parallel and Distributed Systems*, 12(10): 1094–1104, 2001.

[28] Lewei Jin, Kui Zhang, Yongqi Chen, Yifan Zhuo, Renjie Li, Yi Gao, Bowei Yang, Zhengong Cai, and Wei Dong. Distributed LLM serving on consumer-grade GPUs by reconciling computation and communication. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 17633～17642, 2025.

[29] Adam Langley et al. The QUIC transport protocol: Design and Internet-scale deployment. In *Proceedings of the ACM SIGCOMM 2017 Conference*, 2017.

[30] Youhe Jiang et al. HexGen-2: Disaggregated generative inference of LLMs in heterogeneous environment. In *International Conference on Learning Representations (ICLR)*, 2025.

[31] Youhe Jiang et al. Demystifying cost-efficiency in LLM serving over heterogeneous GPUs. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

[32] Xupeng Miao et al. SpotServe: Serving generative large language models on preemptible instances. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.

[33] Bingyang Wu et al. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920*, 2023.

[34] Biao Sun et al. Llumnix: Dynamic scheduling for large language model serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

[35] Hyungjun Oh et al. ExeGPT: Constraint-aware resource scheduling for LLM inference. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.

[36] Zhuohan Li et al. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2023.

[37] Jiangfei Duan et al. MuxServe: Flexible spatial-temporal multiplexing for multiple LLM serving. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[38] Kan Zhu et al. NanoFlow: Towards optimal large language model serving throughput. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2025.

[39] Bingyang Wu et al. LoongServe: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP)*, 2024.

[40] Lianmin Zheng et al. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[41] Foteini Strati et al. DéjàVu: KV-cache streaming for fast, fault-tolerant generative LLM serving. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[42] Yuhan Liu et al. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024.

[43] Mohammad Alizadeh, Shuang Yang, Milad Sharif, Sachin Katti, Nick McKeown, Balaji Prabhakar, and Scott Shenker. pFabric: Minimal near-optimal datacenter transport. In *Proceedings of the ACM SIGCOMM 2013 Conference*, 2013.